

GitHub Projects Custom Fields Automation Guide

A practical guide to GitHub Projects custom fields, organization issue fields, and automation patterns for setting default-like values with GitHub Actions and the GraphQL API.

Prepared May 5, 2026 · Focus: text, number, and single-select defaults; Priority and Effort organization fields; workflow limitations; and troubleshooting form/reporting issues such as blank or “No response” values.

Bottom line. GitHub’s documented Project field UI explains how to create text, number, and single-select fields, but it does not document a native per-field default-value setting for those field types. For reliable defaults, use one of three documented paths: issue-form prefills where the desired value belongs in the issue body, organization issue fields where the value belongs on the issue itself, or GitHub Actions plus the Projects GraphQL API where the value must be written to the Project item after creation.¹²³⁷⁸

How to read this guide. The word “default” is used carefully: native default means GitHub documents a built-in default or prefill behavior; automation default means a workflow assigns a value immediately after item creation; form default means an issue form pre-populates or requires user input before the issue exists.

Recommended architecture

Need	Best-supported GitHub feature	Why this is the right layer
Default issue title, labels, assignees, project routing, issue type	Issue forms top-level keys or issue URL query parameters	These are issue-creation defaults. The issue form docs define top-level keys such as title, labels, assignees, type, and projects; URL parameters can also prefill issue creation data when the user has permission. ⁹¹¹
Default visible issue metadata across an organization	Organization issue fields	Issue fields are defined at the organization level and apply across repositories. GitHub creates Priority, Effort, Start date, and Target date when issue fields are enabled. ⁴
Default project-only metadata such as T-shirt size, score, customer, internal note	GitHub Actions + GraphQL updateProjectV2ItemFieldValue	Projects API mutations can update text, number, date, single-select, and iteration project fields after an item has been added. ⁷
Set status when item is added or issue/PR changes state	Built-in Project workflows	Built-in workflows are documented primarily around Status updates, automatic archiving, and repository auto-add behavior. ⁵⁶

Field type facts and default strategy

Project fields and organization issue fields are related but not identical. Project custom fields live on the Project item; organization issue fields live on the issue and can appear in Projects, subject to field limits. That distinction matters because an issue-form body response is not the same as a Project field value, and setting a Project item field requires the Projects API.⁴⁷

Field	Official behavior	Default strategy
Text Project field	Text fields hold freeform project notes and are filterable with exact-text syntax such as <code>field:"exact text"</code> . ²	No documented UI default. Use GraphQL value <code>{ text: "..." }</code> after <code>addProjectV2ItemById</code> . ⁷
Number Project field	Number fields support comparison and range filters such as <code>></code> , <code>>=</code> , <code><</code> , <code><=</code> , and <code>..</code> range queries. ²	No documented UI default. Use GraphQL value <code>{ number: 3 }</code> for numeric defaults; verify numeric formatting and decimals in the field. ⁷
Single-select Project field	Single-select fields have dropdown options with optional colors/descriptions and can contain up to 50 options. ³	No documented UI default. Query the option ID, then use value <code>{ singleSelectOptionId: "OPTION_ID" }</code> . ⁷⁸
Organization Priority	When issue fields are enabled, GitHub creates Priority with Urgent, High, Medium, and Low options. ⁴	Use when priority should be issue-level metadata shared across projects; customize options if your org uses different terminology.
Organization Effort	When issue fields are enabled, GitHub creates Effort with High, Medium, and Low options. ⁴	Use when effort belongs to issue triage rather than one project's planning model.

Native UI defaults: what GitHub documents

GitHub's field docs describe how to add text, number, and single-select fields in a Project view, how to edit single-select options, and how those fields participate in filters. Those same docs do not describe a Project-field setting named default, default value, initial value, or preselected value for text, number, or single-select Project fields.¹²³

That absence is important for implementation planning. If the desired behavior is "every new project item should automatically have this value," treat it as automation rather than as a field-configuration setting, then set the value with built-in Status workflows when Status is enough or with the GraphQL API when you need arbitrary text, number, or single-select fields.⁵⁷

Issue forms are different from Project fields. GitHub's form schema documents prefilled input and textarea values with the value key and dropdown defaults with a zero-based default index, but submitted issue-form responses are converted to markdown and added to the issue body rather than automatically mapped to Project custom fields.⁹¹⁰

Issue form defaults that are officially supported

Form element	Supported default/prefill	Use it for	Do not confuse with
input	<code>attributes.value</code> pre-fills a single-line text field. ¹⁰	Contact, customer, environment, repo name, expected version.	Project text field value.
textarea	<code>attributes.value</code> pre-fills a multi-line text area. ¹⁰	Template text, checklist prompts, reproduction steps scaffold.	Project text field value.
dropdown	<code>attributes.default</code> sets the preselected option by zero-based index; options must be distinct and cannot include None or n/a when a default is specified. ¹⁰	User-facing category, version, severity choice in the issue body.	Project single-select option ID.
top-level title / labels / assignees / projects / type	Issue forms can set issue defaults such as title, labels, assignees, type, and projects; adding via projects requires the opener to have write permission, otherwise auto-add workflow is recommended. ⁹	Issue creation defaults and routing.	Project item field defaults.

Example: issue form prefilled text and dropdown

```

name: Intake
description: Create a triaged work item.
title: "[Intake]: "
labels: ["triage"]
projects: ["octo-org/1"]
body:
  - type: input
    id: customer
    attributes:
      label: Customer
      value: "Internal"
    validations:
      required: true
  - type: textarea
    id: notes
    attributes:
      label: Notes
      value: |
        Context:
        Expected outcome:
  - type: dropdown
    id: priority
    attributes:
      label: Priority
      options:
        - Low
        - Medium
        - High
      default: 1
    validations:
      required: true

```

This YAML creates issue-body defaults, not Project-field defaults. If a Project single-select field named Priority must be set too, add a workflow that parses or infers the intended value and calls `updateProjectV2ItemFieldValue` with the matching `singleSelectOptionId`.⁷⁸

Organization default fields: Priority and Effort

Organization issue fields are currently in public preview and subject to change. They are designed for structured issue metadata across the organization, not just within one Project.⁴

- **Default fields:** enabling issue fields creates Priority, Effort, Start date, and Target date automatically. Priority is single-select with Urgent, High, Medium, Low; Effort is single-select with High, Medium, Low.⁴
- **Customizable:** these default fields can be renamed, have descriptions/options edited, or be deleted if they do not fit the organization's workflow.⁴
- **Visibility:** fields default to Organization only; Public visibility can expose the field on public repositories, and visibility applies across web UI, API, timeline events, and search suggestions.⁴
- **Pinning:** fields must be pinned to issue types or to Issues without a type to automatically appear in the issue sidebar; otherwise they are available through Add field or in Projects.⁴
- **Limits:** GitHub documents 25 issue fields per organization, 50 options per single-select issue field, 10 pinned fields per issue type, and 50 total fields in a Project including issue fields and system fields.⁴

Use organization issue fields for metadata that should follow the issue across Projects. Use Project custom fields for project-specific planning data where the same issue may need different values in different Projects.

Programmatic defaults with GitHub Actions

The reliable automation pattern is two-step: first add or locate the Project item, then update Project item fields. GitHub's API docs explicitly state that you cannot add and update a Project item in the same call; `addProjectV2ItemById` and `updateProjectV2ItemFieldValue` must be separate operations.⁷

Authentication matters. GitHub's Actions guide states that `GITHUB_TOKEN` is scoped to the repository level and cannot access Projects; it recommends a GitHub App for organization Projects and a classic personal access token with project and repo scopes for user Projects.⁸

For organization Projects, GitHub App permissions must include read and write organization Projects; repository Projects permissions are not a substitute for organization Projects permissions.⁸

Workflow skeleton

```
name: Project defaults

on:
  issues:
    types: [opened, transferred, labeled]
  pull_request:
    types: [opened, ready_for_review]

jobs:
  set-project-defaults:
    runs-on: ubuntu-latest
    steps:
      - name: Generate GitHub App token
        id: app-token
        uses: actions/create-github-app-token@v2
        with:
          app-id: ${vars.APP_ID}
          private-key: ${secrets.APP_PEM}

      - name: Set defaults
        env:
          GH_TOKEN: ${steps.app-token.outputs.token}
          ORGANIZATION: octo-org
          PROJECT_NUMBER: 5
          DEFAULT_CUSTOMER: Internal
          DEFAULT_EFFORT: "3"
          DEFAULT_PRIORITY: Medium
        run: |
          # Query IDs, add/locate the item, then update field values.
          # See the full examples below for the GraphQL pieces.
```

Query project, field, and option IDs

GitHub's Actions example queries the project by organization login and project number, returns the Project ID, field IDs, and single-select option IDs, and extracts them with `jq` before running mutations.⁸

```
gh api graphql -f query='
query($org: String!, $number: Int!) {
  organization(login: $org) {
    projectV2(number: $number) {
      id
      fields(first: 50) {
        nodes {
          ... on ProjectV2Field {
            id
            name
          }
          ... on ProjectV2SingleSelectField {
            id
            name
            options {
              id
              name
            }
          }
        }
      }
    }
  }
}
```

```

    }
  }
}' -f org="$ORGANIZATION" -f number="$PROJECT_NUMBER" > project_data.json

echo "PROJECT_ID=$(jq -r '.data.organization.projectV2.id' project_data.json \
)" >> "$GITHUB_ENV"
echo "CUSTOMER_FIELD_ID=$(jq -r '.data.organization.projectV2.fields.nodes[] \
| select(.name=="Customer") | .id' project_data.json)" >> "$GITHUB_ENV"
echo "EFFORT_FIELD_ID=$(jq -r '.data.organization.projectV2.fields.nodes[] | \
select(.name=="Effort Score") | .id' project_data.json)" >> "$GITHUB_ENV"
echo "PRIORITY_FIELD_ID=$(jq -r '.data.organization.projectV2.fields.nodes[] \
| select(.name=="Priority") | .id' project_data.json)" >> "$GITHUB_ENV"
echo "PRIORITY_OPTION_ID=$(jq -r '.data.organization.projectV2.fields.nodes[] \
| select(.name=="Priority") | .options[] | \
select(.name==env.DEFAULT_PRIORITY) | .id' project_data.json)" >> \
"$GITHUB_ENV"

```

Add the issue or pull request to the Project

The `addProjectV2ItemById` mutation adds the issue or pull request node to the Project and returns the Project item ID, which is the `itemId` required by `updateProjectV2ItemFieldValue`.⁷⁸

```

CONTENT_ID="{ { github.event.issue.node_id || \
github.event.pull_request.node_id }}"

item_id="$(gh api graphql -f query='
mutation($project: ID!, $content: ID!) {
  addProjectV2ItemById(input: { projectId: $project, contentId: $content }) {
    item { id }
  }
}' -f project="$PROJECT_ID" -f content="$CONTENT_ID" --jq \
'.data.addProjectV2ItemById.item.id')"

echo "ITEM_ID=$item_id" >> "$GITHUB_ENV"

```

Set text, number, and single-select defaults

The Projects API supports updating single-select, text, number, date, and iteration Project item fields with `updateProjectV2ItemFieldValue`. Assignees, labels, milestone, and repository are issue or pull request properties and must be changed with their own issue/PR mutations rather than this Project field mutation.⁷

```

gh api graphql -f query='
mutation(
  $project: ID!,
  $item: ID!,
  $customerField: ID!,
  $customerValue: String!,
  $effortField: ID!,
  $effortValue: Float!,
  $priorityField: ID!,
  $priorityOption: String!
) {
  set_customer: updateProjectV2ItemFieldValue(input: {
    projectId: $project
    itemId: $item
    fieldId: $customerField
    value: { text: $customerValue }
  }) { projectV2Item { id } }

  set_effort: updateProjectV2ItemFieldValue(input: {
    projectId: $project
    itemId: $item
    fieldId: $effortField
    value: { number: $effortValue }
  }) { projectV2Item { id } }

  set_priority: updateProjectV2ItemFieldValue(input: {
    projectId: $project
    itemId: $item
    fieldId: $priorityField
    value: { singleSelectOptionId: $priorityOption }
  }) { projectV2Item { id } }
}' -f project="$PROJECT_ID" -f item="$ITEM_ID" -f \
customerField="$CUSTOMER_FIELD_ID" -f customerValue="$DEFAULT_CUSTOMER" \
-f effortField="$EFFORT_FIELD_ID" -f effortValue="$DEFAULT_EFFORT" -f \
priorityField="$PRIORITY_FIELD_ID" -f \

```

```
priorityOption="$PRIORITY_OPTION_ID" --silent
```

Implementation note. The official API guide explicitly shows text and single-select examples and describes number fields as supported; the GraphQL mutation reference also lists number as supported for `updateProjectV2ItemFieldValue`. If your workflow runner reports a GraphQL type error for number variables, inspect the schema in your environment and adjust the variable type to the currently exposed `ProjectV2FieldValue` number scalar.⁷

Built-in workflow automation limitations

GitHub's built-in Project workflows are useful but narrower than a general defaulting engine. The docs describe workflows that update Status when items are added or when issues/PRs close or merge, close issues when Status changes, auto-archive items, and auto-add repository items that match filters.⁵⁶

When a Project initializes, GitHub enables two default workflows: closed issues or pull requests set Status to Done, and merged pull requests set Status to Done.⁵

Built-in capability	What it does	Limitation for defaults
Status updates	Can set Status to values like Todo or Done based on item/project events. ⁵	Best for Status only; not a general mechanism for text, number, or arbitrary single-select defaults.
Auto-add	Adds new or updated repository items that match supported filters; existing matching items are not added when the workflow is enabled. ⁶	Adds items but does not itself set arbitrary custom field defaults.
Auto-add filters	Supports a subset: is, label, reason, assignee, no; most filters support negation. ⁶	Cannot express arbitrary Project-field defaulting rules.
Auto-add counts	Max auto-add workflows are plan-dependent: Free 1, Pro/Team 5, Enterprise Cloud/Server 20. ⁶	May force consolidation or Actions-based routing for many repos/rules.

Troubleshooting

Most defaulting issues come from mixing layers: issue forms create markdown body content, organization issue fields create issue metadata, Project fields create Project item metadata, and built-in workflows mostly update Status or route items. Troubleshoot by first identifying which layer should own the value.

Symptom	Likely cause	Fix
Field shows blank or "No response" in issue body	Issue form field was optional and the user submitted it empty, or a placeholder was mistaken for a value. GitHub's form schema distinguishes placeholder from value and documents required as false by default. ¹⁰	Use <code>attributes.value</code> for actual prefilled text, set <code>validations.required: true</code> where an answer is mandatory, or remove optional prompts that produce noisy blank sections.
Project field remains empty although form has an answer	Issue-form responses are converted to markdown in the issue body; they are not automatically mapped to Project custom fields. ⁹	Add an Actions workflow that parses the issue body or uses labels/types, then calls <code>updateProjectV2ItemFieldValue</code> .
Dropdown default selects wrong option	The form schema default is a zero-based index into the options array. ¹⁰	Count from zero and keep options stable. If options are reordered, update the default index.
Single-select mutation fails	Used option name instead of <code>singleSelectOptionId</code> , or the option belongs to a different field/project. ⁸	Query fields/options in the target Project and pass the option ID, not the display name.

Symptom	Likely cause	Fix
Cannot update Assignees, Labels, Milestone, or Repository through Project mutation	Those are issue/PR properties, not Project item fields. ⁷	Use <code>addLabelsToLabelable</code> , <code>removeLabelsFromLabelable</code> , <code>addAssigneesToAssignable</code> , <code>updateIssue</code> , <code>updatePullRequest</code> , or related mutations.
Workflow cannot access the Project	GITHUB_TOKEN is repository-scoped and the Actions guide says it cannot access Projects. ⁸	Use a GitHub App token for organization Projects or a PAT with project/repo scopes for user Projects.
Auto-add did not import existing issues	Auto-add only adds matching items when they are created or updated after the workflow is enabled. ⁶	Bulk add existing items manually or run a one-time script to add and default them.
Actions job/rate issues at scale	Actions and API calls have rate/queue/runtime limits, including GITHUB_TOKEN's documented repository rate budget. ¹²	Batch field updates, reduce triggers, avoid duplicate workflows, and prefer GitHub App auth for organization-scale automation.

Operational checklist

- Name the layer. Decide whether the value belongs to issue form body, organization issue field, Project field, label, milestone, or assignee.
- Prefer organization issue fields for shared metadata. Priority and Effort are organization-level defaults when issue fields are enabled, and they can appear across Projects within GitHub's field limits.⁴
- Use built-in Status workflow only when Status is enough. Otherwise move to Actions and the API.⁵⁷
- Query IDs dynamically. Store field and option names in workflow variables, but resolve IDs at runtime to avoid hardcoding IDs that change across projects.
- Make workflows idempotent. Check whether an item already exists or whether a field already has a value before overwriting human edits.
- Log the exact IDs used. Mask tokens, but print field names and mutation responses in a safe debug mode.
- Backfill separately. Auto-add does not import existing matching items, so plan a one-time migration for current backlog items.⁶

Decision matrix

If you want...	Use...	Avoid...
Every issue to start with Priority=Medium across the organization	Organization issue fields plus pinned field/default process, or an Actions workflow if you need automatic value assignment.	A Project-only field if the value needs to be visible on the issue everywhere.
Every item in one Project to have Effort Score=3	Actions + <code>updateProjectV2ItemFieldValue</code> number update.	Issue form value alone.
New bugs to enter Project with Status=Todo	Built-in workflow when item added, or Actions if you need broader logic.	A custom single-select named Status unless you deliberately want to bypass GitHub's standard Status workflows.
User to choose severity from issue intake form	Issue form dropdown with default index and required validation.	Assuming that dropdown choice automatically populates a Project single-select.
Existing backlog to receive default values	One-time script using Projects API.	Auto-add workflow alone, because existing matching items are not added when auto-add is enabled. ⁶

Appendix: source map

1. Projects fields: <https://docs.github.com/en/issues/planning-and-tracking-with-projects/understanding-fields>
2. Text & number fields: <https://docs.github.com/en/issues/planning-and-tracking-with-projects/understanding-fields/about-text-and-number-fields>
3. Single-select fields: <https://docs.github.com/en/issues/planning-and-tracking-with-projects/understanding-fields/about-single-select-fields>
4. Organization issue fields: <https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/managing-issue-fields-in-your-organization>
5. Built-in automations: <https://docs.github.com/en/issues/planning-and-tracking-with-projects/automating-your-project/using-the-built-in-automations>
6. Auto-add workflow: <https://docs.github.com/en/issues/planning-and-tracking-with-projects/automating-your-project/adding-items-automatically>
7. Projects API: <https://docs.github.com/en/issues/planning-and-tracking-with-projects/automating-your-project/using-the-api-to-manage-projects>
8. Actions automation: <https://docs.github.com/en/issues/planning-and-tracking-with-projects/automating-your-project/automating-projects-using-actions>
9. Issue forms: <https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/syntax-for-issue-forms>
10. Form schema: <https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/syntax-for-githubs-form-schema>
11. Issue URL prefills: <https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-an-issue>
12. Actions limits: <https://docs.github.com/en/actions/reference/limits>